

Problem Set 6

Due DateOctober 15, 2024 (11:59 PM)
Name **Hyma Jujjuru**
Student ID **110743269**
Collaborators **List Your Collaborators Here**

Contents

Instructions	1
Honor Code (Make Sure to Virtually Sign)	2
1 Quicksort Algorithm	3
1.1 Problem 1 [10 pts]	3
1.2 Problem 2 [20 pts]	4
1.3 Problem 3 [20 pts]	5
1.4 Problem 4 [20 pts]	7
1.5 Problem 5 [30 pts]	8

Instructions

- The solutions **should be typed**, using proper mathematical notation. We cannot accept hand-written solutions. Here's a short intro to $\text{\LaTeX}$.
- You should submit your work through the **class Gradescope page** only (linked from Canvas). Please submit one PDF file, compiled using this $\text{\LaTeX}$ template.
- You may not need a full page for your solutions; pagebreaks are there to help Gradescope automatically find where each problem is. Even if you do not attempt every problem, please submit this document with no fewer pages than the blank template (or Gradescope has issues with it).
- You are welcome and encouraged to collaborate with your classmates, as well as consult outside resources. You must **cite your sources in this document**. **Copying from any source is an Honor Code violation. Furthermore, all submissions must be in your own words and reflect your understanding of the material.** If there is any confusion about this policy, it is your responsibility to clarify before the due date.
- Posting to **any** service including, but not limited to Chegg, Reddit, StackExchange, etc., for help on an assignment is a violation of the Honor Code.

- You **must** virtually sign the Honor Code (see Section). Failure to do so will result in your assignment not being graded.

Honor Code (Make Sure to Virtually Sign)]

Problem HC. • My submission is in my own words and reflects my understanding of the material.

- Any collaborations and external sources have been clearly cited in this document.
- I have not posted to external services including, but not limited to Chegg, Reddit, StackExchange, etc.
- I have neither copied nor provided others solutions they can copy.

Agreed (signature here). I agree to the above, Hyma Jujjuru.

□

1 Quicksort Algorithm

1.1 Problem 1 [10 pts]

Problem 1. Briefly explain when Quicksort achieves its **best-case** running time. Then, write the recurrence relation for the best-case scenario of Quicksort and solve it to determine the runtime. Make sure to provide **both the recurrence and the final runtime**. You may use either the Unrolling Method or the Recursion Tree Method.

Proof. **Quicksort Pseudocode:**

```
procedure QUICKSORT(A, p, r)
  if p < r then
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q-1)
    QUICKSORT(A, q+1, r)
```

```
procedure PARTITION(A, p, r)
  x = A[r]
  i = p - 1
  for j = p, j ≤ r - 1, j ++ do
    if A[j] ≤ x then
      i++
      swap(A[i], A[j])
  swap(A[i+1], A[r])
  return i+1
```

The **best-case** for QuickSort is when the pivot chosen is the median. This means the array is split into two parts each time. (Note: we choose the pivot to be the last element in the list)

Recurrence Relation

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1 \\ 2T(\frac{n}{2}) + \Theta(n) & \text{if } n > 1 \end{cases} \quad (1)$$

Note: $\Theta(1) = b$ and $\Theta(n) = cn$

Also, $\frac{n-1}{2}$ (if the pivot is ignored then there are $n-1$ elements) and $\frac{n}{2}$ have the same asymptotic runtime complexity. (Told by Professor Lijun)

$i = \log_2 n$ because it takes $\log_2 n$ levels to reach the base case

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + cn \\ &= 2\left[2T\left(\frac{n}{2^2}\right) + c\frac{n}{2}\right] + cn = 2^2T\left(\frac{n}{2^2}\right) + 2cn \\ &= 2^2\left[2T\left(\frac{n}{2^3}\right) + c\frac{n}{4}\right] + 2cn = 2^3T\left(\frac{n}{2^3}\right) + 3cn \\ &= \dots \\ &= 2^iT\left(\frac{n}{2^i}\right) + i \cdot cn \\ &= 2^{\log_2 n}T\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n \cdot cn \\ &= nT\left(\frac{n}{n}\right) + \log_2 n \cdot cn \\ &= nT(1) + cn \log_2 n \\ &= n(b) + cn \log_2 n \end{aligned}$$

Final Runtime: $T(n) = \Theta(n \log n)$.

□

1.2 Problem 2 [20 pts]

Problem 2. Briefly explain when Quicksort achieves its **worst-case** running time. Then, write the recurrence relation for the worst-case scenario of Quicksort and solve it to determine the runtime. Make sure to provide **both the recurrence and the final runtime**. You may use either the Unrolling Method or the Recursion Tree Method.

Answer. **Quicksort Pseudocode:**

```
procedure QUICKSORT(A, p, r)
  if p < r then
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q-1)
    QUICKSORT(A, q+1, r)
```

```
procedure PARTITION(A, p, r)
  x = A[r]
  i = p - 1
  for j = p, j ≤ r - 1, j ++ do
    if A[j] ≤ x then
      i++
      swap(A[i], A[j])
  swap(A[i+1], A[r])
  return i+1
```

The **worst-case** for QuickSort is when the pivot chosen is the last element or the first element of the list. In other words, the list given is already sorted (ascending order) or reversely sorted (descending order) because the algorithm needs to traverse all n elements to sort all the elements either all to the left or all to the right, not both. This causes the algorithm to run $n - 1$ times because it takes that many times to traverse the entire array while placing one element (the pivot) in the correct spot each time. (Note: we choose the pivot to be the last element in the list)

Recurrence Relation:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1 \\ T(n-1) + \Theta(n) & \text{if } n > 1 \end{cases} \quad (2)$$

Note: $\Theta(n) = cn$ and $\Theta(1) = c$

$i = n - 1$ because it takes $n - 1$ levels to reach the base case because $n - i = 1 \rightarrow i = n - 1$.

$$\begin{aligned} T(n) &= T(n-1) + c(n) \\ &= T(n-2) + c(n-1) + c(n) \\ &= T(n-3) + c(n-2) + c(n-1) + c(n) \\ &= \dots \\ &= T(n-i) + \sum_{i=1}^n c(n) \\ &= T(n-(n-1)) + n \cdot c(n) \\ &= T(1) + c(n^2) \\ &= c(1) + c(n^2) \end{aligned}$$

Final Runtime: $T(n) = \Theta(n^2)$.

□

1.3 Problem 3 [20 pts]

Problem 3. Consider a variation of Quicksort where the pivot selection consistently results in uneven splits. Specifically, at each recursive step, the array is divided into two parts: one with $n/3$ elements and another with $2n/3$ elements. Briefly explain how Quicksort would perform in this scenario. Then, write the recurrence relation for this scenario. And, solve it to determine the runtime. Make sure to provide **both the recurrence and the final runtime**. You may use either the Unrolling Method or Tree Method.

Answer. **Original Quicksort Pseudocode:**

```

procedure QUICKSORT( $A, p, r$ )
  if  $p < r$  then
     $q = \text{PARTITION}(A, p, r)$ 
    QUICKSORT( $A, p, q-1$ )
    QUICKSORT( $A, q+1, r$ )

```

```

procedure PARTITION( $A, p, r$ )
   $x = A[r]$ 
   $i = p - 1$ 
  for  $j = p, j \leq r - 1, j++$  do
    if  $A[j] \leq x$  then
       $i++$ 
      swap( $A[i], A[j]$ )
  swap( $A[i+1], A[r]$ )
  return  $i+1$ 

```

Quicksort Variation Explanation:

Since the pivot selection results in an uneven split such that the array splits into two parts: one with $\frac{2n}{3}$ elements and one with $\frac{n}{3}$ elements which is repeated at each recursive step.

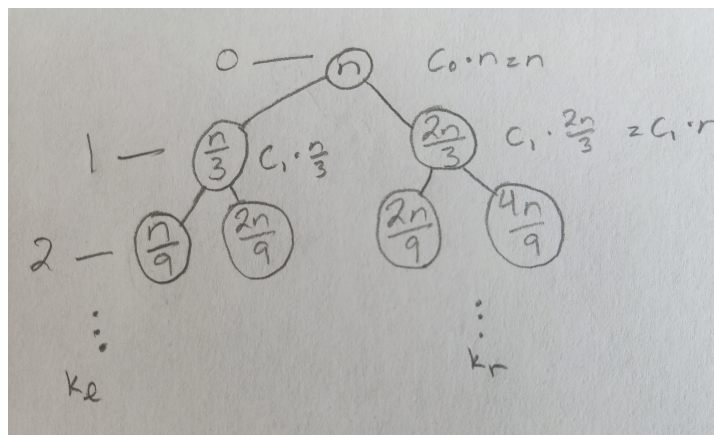
Recurrence Relation:

The time complexity of the partition is $\Theta(n)$ as seen from the pseudocode since the for loop runs in linear time. At each recursion call the array is split into $\frac{2n}{3}$ and $\frac{n}{3}$, so the recursion relation is as follows:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1 \\ T(\frac{2n}{3}) + T(\frac{n}{3}) + \Theta(n) & \text{otherwise} \end{cases} \quad (3)$$

Note: $\Theta(n) = cn, \Theta(1) = b$

$$T(n) = T(\frac{2n}{3}) + T(\frac{n}{3}) + cn$$



For the left branch, let i = the level of the tree and k_l = the level of the base case of the left branch

$$\frac{n}{3^i} \longrightarrow \frac{n}{3^{k_l}} = 1 \longrightarrow k_l = \log_3 n \quad (4)$$

For the right branch, let i = the level of the tree and k_r = the level of the base case of the right branch

$$\left(\frac{2}{3}\right)^i n \longrightarrow \left(\frac{2}{3}\right)^{k_r} n = 1 \longrightarrow k_r = \log_{3/2} n \quad (5)$$

Since the total work done at each level is cn which is $\Theta(n)$, the run time complexity would be $\Theta(n \log_3 n + n \log_{3/2} n) = \Theta(n \log n)$

Final Runtime: $T(n) = \Theta(n \log n)$

□

1.4 Problem 4 [20 pts]

Problem 4. Suppose that we modify PARTITION so that it chooses the minimum element as the pivot that occur in nodes of the recursion tree whose depth is divisible by 2, and it chooses the median element as the pivot in calls that occur in nodes **all** other depth of this recursion tree.

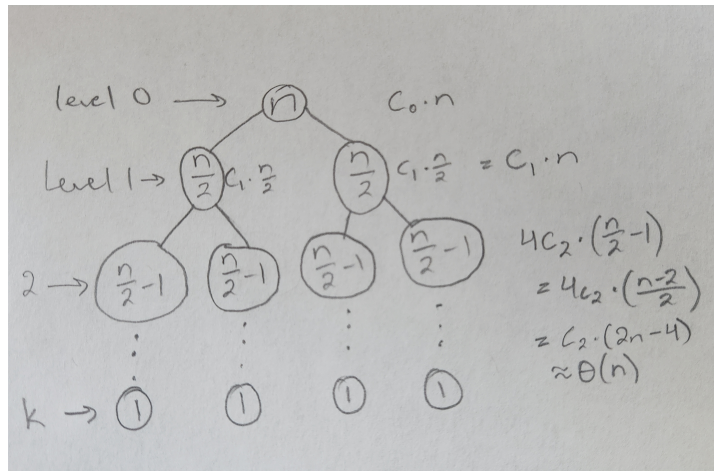
Assume that the running time of this modified PARTITION is still $\Theta(n)$ on any subarray of length n . And the root of a recursion tree starts at level 0 (which is divisible by 2), its children are at level 1, etc.

For example, the modified PARTITION chooses the median element at the root of the recursion tree, in the next layer of the recursion tree it chooses the minimum element, and so on.

Write a recurrence relation for the running time of modified QUICKSORT given an array n distinct elements. **The final recurrence relation is sufficient, no need to solve it asymptotically.**

Answer. The modified PARTITION:

Drawing a recursion tree, we choose the pivot to be the median element at level 0 and the minimum element at level 1, median at level 2, minimum at level 3 and so on. This is displayed in the image below.



Also, the **best-case** scenario of QUICKSORT/PARTITION is when the chosen pivot is the median, and the **worst-case** scenario is when the chosen pivot is either the minimum or maximum element of the given array. Translating that into this problem shows that at level 0, we consider the best case meaning the children of the node are $2(\frac{n}{2})$. At level 1, we consider the worst case, the children of each $\frac{n}{2}$ is $\frac{n}{2} - 1$. And the pattern follows that level 2 = best case, level 3 = worst case, and so on. Now we can write the recurrence relation as follows.

Recurrence Relation:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1 \text{ (base case)} \\ 2T(\frac{n}{2}) + \Theta(n) & \text{if } n \% 2 = 0 \text{ and } n > 1 \\ T(n-1) + \Theta(n) & \text{if } n \% 2 = 1 \text{ and } n > 1 \end{cases} \quad (6)$$

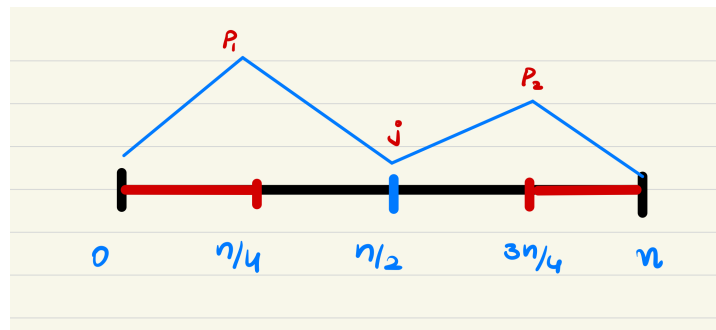
□

1.5 Problem 5 [30 pts]

Problem 5. Captain Algos needs your help analyzing the flight path of her drone over a mountain range. The drone's altitude at different points is recorded in an array, and it forms a peak.

A peaked array has the property that the subarray $A[1..i]$ has the property that the altitude increases steadily, i.e. $A[j] < A[j + 1]$ for $1 \leq j < i$, and after reaching the highest point at index i , the altitude starts decreasing in the subarray $A[i..n]$ has the property that $A[j] > A[j + 1]$ for $i \leq j < n$. Using a flight simulator, she notes the following peaked array $A = [7, 8, 10, 15, 16, 23, 19, 17, 4, 1]$, as an example.

1. She needs to find the highest point to ensure the drone has enough battery and can handle the pressure changes. Write a recursive algorithm (pseudocode) that takes asymptotically **sub-linear time** to find the maximum element of A . Please write the complete pseudocode in algorithmic style. Use `algorithm` package in latex. Please give a brief explanation of different aspects of your code.
2. Prove that your algorithm is correct. (Hint: can we use induction/ Loop invariance).
3. **Bonus Points** [40 pts]:
 - (a) Now consider a generalization, in which the array contains k peaks, i.e., it contains k subarrays. Let $k = 2$, can we prove that our algorithm in part (a) can fail on such an input. Again, remember we need to find the highest peak in the flight path. [10 pts]
 - (b) Suppose that $k = 2$ and we can guarantee that neither peak is closer than $n/4$ positions to the middle of the array, and that the **joining point (j)** of the two singly-peaked subarrays lays in the middle half of the array. Now write an algorithm that returns the maximum element of A in sublinear time. Based on the previous parts give a logical reason why your algorithm is correct, give a recurrence relation for its running time, and solve for its asymptotic behavior. [30 pts]



Answer. 1. Pseudocode: To find the highest peak we can use binary-search which has a runtime complexity of $O(\log n)$

Algorithm 1 Find the highest peak in the flight path

```

1: procedure FINDHIGHESTPEAK( $A, left, right$ )
2:   if  $right < left$  then return -1
3:   if  $left = right$  then return  $A[left]$ 
4:    $mid = (left + right) // 2$ 
5:   if  $A[mid] < A[mid + 1]$  then
6:     return FINDHIGHESTPEAK( $A, mid + 1, right$ )           ▷ Peak must be in right half
7:   else
8:     return FINDHIGHESTPEAK( $A, left, mid$ )               ▷ Peak must be in left half

```

2. Proof of Correctness Using Induction/Loop Invariant:

Loop Invariant: At each recursive call, there is at least one peak in the subarray $A[left, \dots, right]$

Base Case: when $left = right$, we have narrowed down to the index of the highest peak.

Inductive Step: loop invariant holds for a subarray of size n .

Consider the case where $right - left > 0$. The algorithm calculates the midpoint using this formula:
 $mid = \frac{right + left}{2}$

We will consider three cases during each recursion call:

- **Case 1:** $A[mid] < A[mid + 1]$

If the midpoint's right neighbor has a higher value than the midpoint then it indicates an increasing sequence to the right of the midpoint $\rightarrow$ the peak is in the subarray $A[mid + 1, \dots, right]$.

- **Case 2:** $A[mid] > A[mid + 1]$

If the midpoint's right neighbor has a lower value than the midpoint then it indicates a decreasing sequence to the left of the midpoint $\rightarrow$ the peak is in the subarray $A[left, \dots, mid]$.

- **Case 3:** $A[mid] = A[mid + 1]$ or $A[mid] = A[mid - 1]$

If the midpoint is equal to either of its neighbors, we can move in the direction of larger values to find the peak. The algorithm recursively chooses the left half subarray, $A[left, \dots, mid]$.

After all recursion calls are done, the loop invariant holds for all three cases, as there exists at least one peak in each case.

3. Bonus Points:

- (a) Counterexample with two peaks for the algorithm produced in part 1:

$A = [2, 4, 12, 5, 1, 3, 6, 7, 4, 1]$

The two peaks in the array are: $A[2] = 12$ and $A[7] = 7$

Initial call: $left = 0, right = 9, mid = (0 + 9) // 2 = 4$

$A[4] < A[5] (1 < 3) \rightarrow findHighestPeak(A, 5, 9)$

2^{nd} call: $left = 5, right = 9, mid = (5 + 9) // 2 = 7$

$A[7] > A[8] (7 > 4) \rightarrow findHighestPeak(A, 5, 7)$

3^{rd} call: $left = 5, right = 7, mid = (5 + 7) // 2 = 6$

$A[6] < A[7] (6 < 7) \rightarrow findHighestPeak(A, 7, 7)$

4^{th} call: $left = 7, right = 7 \rightarrow return A[left] = A[7] = 7$

While the algorithm finds a peak in the flight's path (7), it fails to find the highest peak in the flight's path (12).

(b) **Maximum Element of A in Sublinear Time where A has 2 peaks:**

Algorithm 2 Find the joining point, j

```
1: procedure FINDJ(A, left, right)
2:   if right < left then return -1
3:   if left = right then return A[left]
4:   mid = (left + right)//2
5:   if A[mid] > A[mid + 1] then
6:     return FINDJ(A, mid+1, right)           ▷ Peak must be in right half
7:   else
8:     return FINDJ(A, left, mid)             ▷ Peak must be in left half
```

Algorithm 3 Find the Maximum Element of A

```
1: procedure FINDMAX(A)
2:   n = A.length
3:   if n == 0 then
4:     return -1
5:   left = n//4
6:   right = 3n//4
7:   j = FINDJ(A, left, right)
8:   peak1 = FINDHIGHESTPEAK(A, 0, j)
9:   peak2 = FINDHIGHESTPEAK(A, j + 1, A.length)
10:  if peak1 > peak2 then
11:    return peak1
12:  else
13:    return peak2
```

Why New Algorithm is Correct:

The joining point, j, is the point where we can split the array into two subarrays that represent a peak each. The algorithm for finding j is the same as the algorithm findHighestPeak, but we are finding the lowest point between the indices $\frac{n}{4}$ and $\frac{3n}{4}$ where n is the length of the array. (The indices are set based on the conditions stated in the problem).

The algorithm findHighestPeak finds the highest peak in the given array so passing in (0,j) and (j,n) to that function finds the peak in each of those intervals. Then we can compare both peaks to find the higher peak.

Recursion Relation of FINDHIGHESTPEAK:

Since the algorithm runs only one recursive call instead of both,

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1 \\ T(\frac{n}{2}) + \Theta(1) & \text{if } n > 1 \end{cases} \quad (7)$$

Runtime Complexity of FINDHIGHESTPEAK:

Note: $\Theta(1) = c$ and $2^i = n \longrightarrow i = \log_2 n$

$$\begin{aligned}
T(n) &= T\left(\frac{n}{2}\right) + c \\
&= [T\left(\frac{n}{4}\right) + c] + c = T\left(\frac{n}{4}\right) + 2c \\
&= [T\left(\frac{n}{8}\right) + c] + 2c = T\left(\frac{n}{8}\right) + 3c \\
&= \dots \\
&= T\left(\frac{n}{2^i}\right) + ic \\
&= T\left(\frac{n}{2^{\log_2 n}}\right) + c \cdot \log_2 n \\
&= T\left(\frac{n}{n}\right) + c \cdot \log_2 n \\
&= T(1) + c \cdot \log_2 n \\
&= c + c \cdot \log_2 n \\
&= c(1 + \log_2 n)
\end{aligned}$$

The runtime complexity of findHighestPeak: $T(n) = \Theta(\log n)$

Recursion Relation and Runtime Complexity of findMax:

Note: There is no recursion in the findMax algorithm but does call an outside function which has a runtime complexity of $\Theta(\log n)$ each as calculated above. The findJ function is the same algorithm as findHighestPeak function with one modification ($A[mid] > A[mid + 1]$ instead of $A[mid] < A[mid + 1]$) so it has the same runtime complexity.

$$\begin{aligned}
T(n) &= 3\Theta(\log n) + \Theta(1) \\
&= \Theta(\log n)
\end{aligned}$$

The final runtime complexity is: $T(n) = \Theta(\log n)$

□